DBSCAN on Resilient Distributed Datasets

Irving Cordova
Department of Computer Science
San José State University
San José, California, U.S.A.
irving@irvingc.com

Teng-Sheng Moh
Department of Computer Science
San José State University
San José, California, U.S.A.
teng.moh@sjsu.edu

***Abstract*—DBSCAN is a well-known density-based data clustering algorithm that is widely used due to its ability to find arbitrarily shaped clusters in noisy data. However, DBSCAN is hard to scale which limits its utility when working with large data sets. Resilient Distributed Datasets (RDDs), on the other hand, are a fast data-processing abstraction created explicitly for in-memory computation of large data sets. This paper presents a new algorithm based on DBSCAN using the Resilient Distributed Datasets approach: RDD-DBSCAN. RDD-DBSCAN overcomes the scalability limitations of the traditional DBSCAN algorithm by operating in a fully distributed fashion. The paper also evaluates an implementation of RDD-DBSCAN using Apache Spark, the official RDD implementation.**

***Keywords*—DBSCAN; Apache Spark; data clustering; parallel systems; data partition; Resilient Distributed Datasets; MapReduce**

I. Introduction

We live in a world that is becoming more and more connected. Smart phones collect information about every aspect of our everyday lives and store this information in centralized locations. Millions of small devices are deployed throughout the globe collecting information on weather conditions, the signals generated by different machines and on human activity in general. The amount of data being generated and stored every day is staggering and continues to grow every day.

The information collected by these systems can be used in many different ways, from predicting possible failures to recommending the best solutions to different problems based on previous experiences. However, when the amount of data gets very large, the difficulty of deriving useful conclusions from the data increases. A popular approach to overcome this difficulty is machine learning, in specific, clustering algorithms. Clustering algorithms ameliorate the complexity of the data by grouping similar data into groups, or clusters, which can then be more readily analyzed.

Among clustering algorithms, Density-based Spatial Clustering of Applications with Noise (DBSCAN) is one of the most widely used. Its popularity stems from the benefits that DBSCAN provides over other clustering algorithms: the ability to discover arbitrarily shaped clusters, robustness in the presence of noise and simplicity in its input parameters (unlike other clustering algorithms, DBSCAN does not require the user to enter the number of clusters to be found prior to execution). DBSCAN is also popular with researchers: it has been heavily cited in many publications and in 2014 DBSCAN was awarded the 2014 SIGKDD test of time award [1].

DBSCAN was originally developed to work in the context of databases running in a single machine [2]. Since machine learning, including clustering, is especially useful in large data sets, it comes as no surprise that there has been a lot of research into enabling DBSCAN to escape out of its single machine origins. Many papers have been published with different approaches for the parallel execution of DBSCAN; some describe a master-slave setup [3], [4], while some others propose a fully distributed parallel algorithm [5]. There is one particular area of research which is of special interest in the context of processing large amounts of data though, the research on scaling DBSCAN through MapReduce.

MapReduce was introduced in 2004 in a seminal paper published by J. Dean and S. Ghemawat [6]. The paper introduced a shared-nothing architecture that allowed the parallel processing of a very large amount of data. The MapReduce paradigm introduced by the paper is now widely used in the industry and is at the heart of the data processing pipelines of many of the companies behind the previously mentioned smart phones and connected devices [7].

Since MapReduce is so widely used, an adaptation of DBSCAN for MapReduce becomes specially useful. From the many papers on this subject, B. R. Dai, and I. C. Lin in [8] and Y. He, et al. in [9] have both proposed variations of DBSCAN that allow the algorithm to run on top of the Apache Hadoop framework, the most popular implementation of the MapReduce paradigm.

One of the big drawbacks of Hadoop's implementation of MapReduce, is that the only communication that can happen between data-processing steps, in a data-processing pipeline, is through the file system. This communication restriction introduces a very high cost, in the form on latency, on top of the already costly computation. This cost is acceptable for pipelines that only do transformations on the data. However, most machine learning algorithms (including DBSCAN) perform many iterations over the same set of data, with each step also depending on results on the previous steps. This characteristic of machine learning algorithms has made those algorithms poor performers on the Hadoop platform [10]. M. Zaharia, et al. noticed this drawback and provided a solution in the form of an abstraction for in-memory computing: Resilient Distributed Datasets (RDDs).

978-1-4673-7813-0/15/$31.00 ©2015 IEEE

Authorized licensed use limited to: UEL - Universidade Estadual de Londrina. Downloaded on October 21,2026 at 19:56:14 UTC from IEEE Xplore. Restrictions apply.

Like MapReduce with Hadoop, RDDs have a dominant implementation that is widely used: Apache Spark. Apache Spark is an implementation of the RDDs abstraction that provides a programming framework that allows the programmer to process large amounts of data and also to take advantage of volatile memory in order to speed-up computation. Thus, machine learning algorithms perform much faster on Apache Spark than on the Hadoop platform. Is it possible to adapt DBSCAN to run fully in parallel under Apache Spark? In this paper we answer this question positively by:

- Describing a modified DBSCAN algorithm that takes full advantage of Apache Spark's parallel capabilities and obtains the same results as the original algorithm.
- Implementing the modified algorithm in the Scala programming language and running it on top of Apache Spark.
- Proving experimentally that the modified algorithm scales as the amount of nodes available to Apache Spark for computation increases.
- Listing the limitations of the given approach and places in which it can be improved.

II. Background

A. Distributed Computing

Traditionally there have been two different approaches for processing large amounts of data. The first approach, when tasked with ever increasing amounts of data, increases the *processing power* of the particular machine with the task of processing data. This approach is commonly referred to as scaling vertically. The second approach, on the other hand, instead of increasing the power of a single machine, increases the *number* of machines that are tasked with the processing of the data. The second approach is commonly referred to as scaling horizontally [11].

In the last few years, scaling horizontally has gained prominence as the preferred approach to processing large amounts of data. The main cause behind this preference is the ever decreasing costs of personal computers, along with their ever increasing computing power [12]. The decrease of costs and the increase of power have made it possible to obtain the same amount of computing power from several cheap personal computers working together, than from a single powerful, but *expensive*, machine. Hence, most companies that are interested in processing data have moved to a horizontally scaling solution.

The move to a horizontally scaling solution, however, brings with it its own set of complexities. As the number of discrete computing elements increases, so does the chances of failure of any single element, increasing the difficulty of achieving reliable computing. In a similar fashion, task allocation also becomes more complex. Ideally, every computing task should be distributed equally throughout the available computing elements, in practice, however, this distribution is difficult. The difficulty arises from the fact that data is not always present in all computing nodes, and that not all computation tasks can be split evenly.

These complexities bring us to MapReduce. MapReduce is a paradigm for processing large amounts of data in a horizontally scalable solution. The main benefit of MapReduce is that it provides a set of operations that allow the user to perform large-scale computations, without worrying about the complexities of distributing the computation throughout the cluster, or worrying about how to recover the computation in the case of failure. MapReduce manages those complexities for the user. With those characteristics, it is no surprise that MapReduce has become the dominant paradigm in horizontally scaling solutions that process large amounts of data [7].

B. Resilient Distributed Datasets

One of the drawbacks of the MapReduce paradigm is that it does not provide an efficient way to implement algorithms that have to perform multiple passes over the same data. The inefficiency stems from the fact that under MapReduce operations on data have to be performed by a combination of MapReduce's three main phases: *map*, *shuffle* and *reduce*.

In the *map* phase, each data unit is transformed and assigned a key (the key may be the same for multiple data units). In the *shuffle* phase, the data is partitioned according to its key and shipped to the reducers. Finally, in the *reduce* phase, the reducers generate some output for each group of data that shares the same key.

Algorithms that make multiple passes over the same data must then allow for the data to be read by the mapper, wait for the reducer to complete, have a different mapper re-read the same data and then, perform the subsequent steps. In other words, all communication between the different steps of the algorithm is done through the data store. Since the data store is usually physical media, the performance of an iterative algorithm becomes worse as the number of passes over the same data increases.

M. Zaharai et al. noticed that, while MapReduce effectively provides an abstraction on top of the computing resources of a cluster [10], iterative algorithms, in order to achieve reasonable levels of performance, need to also manage another of the cluster's resources: memory. Utilizing memory effectively allows iterative algorithms to avoid the performance penalties of reading data from the data store for each of their steps, since the data is already present in memory.

M. Zaharai, et al. proposed Resilient Distributed Datasets (RDDs) as a solution to the shortcomings of MapReduce. RDDs, as defined by its creators, are an abstraction that "enables efficient data reuse in a broad range of applications" [10]. RDDs allow for algorithms to do coarse operations over data (e.g. map, filter, reduce) while also allowing for algorithms to manage the persistence of data in memory.

The main contribution of RDDs, over other approaches to memory management in a cluster, is that the set of operations exposed by RDDs allows for RDDs to provide efficient fault tolerance. While other cluster memory management approaches utilize data replication (which is expensive), RDDs keep track of data *lineage*. The benefit of RDDs' approach is that if data is lost for any reason, the lineage of the data can

Authorized licensed use limited to: UEL - Universidade Estadual de Londrina. Downloaded on October 21,2026 at 19:56:14 UTC from IEEE Xplore. Restrictions apply.

be tracked, and the lost data can be recomputed. Thus, RDDs allow for fault tolerance without the costs of data replication.

In light of these benefits, it is clear that RDDs provide a better environment for iterative algorithms than MapReduce. Many machine learning algorithms, including many clustering algorithms, fall under the iterative algorithms umbrella. Providing adaptations of such algorithms that run under the RDDs abstraction would allow for those algorithms to more efficiently process large amounts of data.

C. The DBSCAN Algorithm

As mentioned before, clustering algorithms are used to reduce data sets into groups, or clusters, which can be more readily analyzed and reasoned about. These clusters can then be used to make predictions about new data, or to find previously unnoticed connections among existing data. That makes clustering algorithms useful in a variety of fields, from machine learning, to data mining, to image analysis.

For a given data set, there are multiple ways to find commonalities between the data in order to generate clusters. This characteristic of clustering has led to the development of several clustering algorithms. Each one of those algorithms uses different criteria to form clusters from the data. Among the most widely used algorithms for clustering we can find K-Means and DBSCAN.

DBSCAN is a density-based clustering algorithm. Density-based clustering algorithms define a cluster as an area that has a higher data density than its surrounding area. In DBSCAN density is measured by analyzing whether a point has at least a minimum number of points ($MinPts$) inside a given radius (ϵ). That is, if the ϵ neighborhood (points within ϵ) of a given point has exceeded a particular density threshold ($MinPts$) that given point, and its neighbors, form a cluster. The DBSCAN algorithm is described in pseudo code in Algorithm 1.

There are a few key concepts that stem from the definition of DBSCAN that bear further explanation:

- $N_\epsilon(p)$: for a point $p \in X$, its ϵ neighborhood is defined as $N_\epsilon(p) = \{x \in X | dist(x,p) \leq \epsilon\}$ where $dist$ is the distance function.
- *core point*: a point $p \in X$ is a core point if $|N_\epsilon(p)| \geq MinPts$
- *directly density reachable*: a point $p \in X$ is directly density reachable from $x \in X$ if $p \in N_\epsilon(x)$ and x is a *core point*.
- *density reachable*: a point $y \in X$ is density reachable from $x \in X$ if there is a chain of points $p_1, p_2, \ldots, p_n$ where $y = p_1$ and $x = p_n$ and each $p_n \neq y$ is directly density reachable from p_{n-1}
- *density connected*: a point $p \in X$ is density connected to $q \in X$ if there is a point $x \in X$ such that p, q are density reachable from x.
- *cluster*: C is a cluster in X if $C \subset X$ and for any $p, q \in C$, p and q are density connected.
- *border point*: p is a border point, if $p \in$ cluster C and p is not a core point.
- *noise point*: p is a border point if it does not belong to any cluster.

Thus, after DBSCAN completes, all points in the input set of points will be classified as either a core point, a border point or noise. Furthermore, all core points and all border points will be assigned a cluster identifier, indicating the cluster to which they belong.

Algorithm 1 The DBSCAN algorithm

Input: A set of points $X = \{p_1, p_2, \ldots, p_n\}$, the distance threshold ϵ, and the minimum number of points required for a cluster $MinPts$.

Output: A set of labeled points $X = \{p_1, p_2, \ldots, p_n\}$, where each point has a flag corresponding to one of CORE, BORDER or NOISE and in the case of the flag being CORE or BORDER a corresponding cluster identifier.

```
1:  clusterIdentifier ← next available cluster identifier
2:  foreach unvisited point p ∈ X do
3:    mark p as visited
4:    N ← GETNEIGHBORS(p, ε)
5:    if |N| < MinPts then
6:      p.flag ← NOISE
7:    else
8:      p.clusterIdentifier ← clusterIdentifier
9:      p.flag ← CORE
10:     foreach p' ∈ N do
11:       if p' is not visited then
12:         mark p' as visited
13:         N' ← GETNEIGHBORS(p', ε)
14:         if |N'| ≥ MinPts then
15:           p'.flag ← CORE
16:           N ← N ∪ N'
17:         else
18:           p'.flag ← BORDER
19:         end if
20:       end if
21:       if p' does not belong to any cluster then
22:         p'.clusterIdentifier ← clusterIdentifier
23:         p'.flag ← BORDER
24:       end if
25:       clusterIdentifier ← next cluster identifier
26:     end for
27:   end if
28: end for
```

III. DBSCAN USING RESILIENT DISTRIBUTED DATASETS

A. Overview

Despite all of its virtues, the DBSCAN algorithm does not lend itself well to the set of operations that are enabled by RDDs: coarse-grained transformations that are applied in parallel to many data items [10]. Although DBSCAN itself does not require a specific starting point or a sequential order, once a point has been selected for processing and the point has entered the region growing phase (Line 10 through 26 in Algorithm 1) no other points can enter the same region. This

Authorized licensed use limited to: UEL - Universidade Estadual de Londrina. Downloaded on October 21,2026 at 19:56:14 UTC from IEEE Xplore. Restrictions apply.

exclusion effectively limits the amount of parallel processing that can be done for the data set and limits the ability of RDDs to effectively execute DBSCAN.

There have been many attempts to parallelize DBSCAN [4], [13]. From those attempts, the research in parallelizing DBSCAN through MapReduce is specially interesting in the case of RDDs. That is because both RDDs and MapReduce expose a similar set of coarse operation on top of data: transformations and reductions, so it follows that the research done in MapReduce can be applied to RDDs.

In this paper we introduce a new algorithm named RDD-DBSCAN. Our algorithm builds on the algorithm presented in [9], which parallelized DBSCAN using MapReduce. Our algorithm follows the same general outline from that paper, but introduces modifications that allow our algorithm to take advantage of the RDDs abstraction's ability to manage memory, improving the algorithm's efficiency. RDD-DBSCAN is described in Algorithm 2.

Algorithm 2 The RDD-DBSCAN algorithm

Input: A set of points $X = \{p_1, p_2, \ldots, p_n\}$, the distance threshold ϵ, the minimum number of points required for a cluster $MinPts$ and the maximum number of points per worker $MaxPoints$.

Output: A set of labeled points $X = \{p_1, p_2, \ldots, p_n\}$, where each point has a flag corresponding to one of CORE, BORDER or NOISE and in the case of the flag being CORE or BORDER a corresponding cluster identifier.

1: $labeledPoints \leftarrow \emptyset$
2: $BR \leftarrow findMinimumBoundingRectangle(X)$
3: $P \leftarrow EvenlyPartition(BR, 2\epsilon, MaxPoints)$
4: **foreach** $partition \in P$ **do**
5: $\quad partition \leftarrow partitionexpandedBy(\epsilon)$
6: $\quad points \leftarrow p_n \in partition$
7: $\quad labeledPoints \cup DBSCAN(points, \epsilon, MinPts)$
8: **end for**
9: $aliases \leftarrow IdentifyAliases(P, labeledPoints, \epsilon)$
10: $clusters \leftarrow$ all unique clusters in $labeledPoints$
11: $RelabelPoints(aliases, clusters, labeledPoints)$

RDD-DBSCAN takes as its input the same parameters as the traditional DBSCAN algorithm: ϵ and $MinPoints$ and introduces a new input parameter: $MaxPoints$. RDD-DBSCAN consists of three phases: data partitioning, local clustering and global labeling.

In the data partitioning phase, RDD-DBSCAN starts by first creating a bounding rectangle B, which spans the whole data set which we wish to cluster. The rectangle B is then recursively split into two smaller rectangles, with each smaller rectangle containing roughly half the data points of its parent rectangle. The process stops when the number of data points contained in any given rectangle is smaller than $MaxPoints$ or when any given rectangle is smaller than 2ϵ. The size of each of the final rectangles is then increased by ϵ for reasons that are described in more detail in section III-B. For every rectangle, the data points contained inside the rectangle constitute a partition. Note that this means that some data points may belong to more than one partition.

After the partitions are found, RDD-DBSCAN enters the local clustering stage. In the local clustering stage, RDD-DBSCAN performs local clustering for each partition using the traditional DBSCAN algorithm. In order to efficiently perform the local clustering, RDD-DBSCAN will load all the data points for a given partition into memory. Only then, will the local clustering be done. Once the local clustering completes, RDD-DBSCAN makes use of the RDDs abstraction's data management operations to persist the results of the clustering into memory.

Since RDD-DBSCAN performs the clustering per individual partition, and each partition does not have knowledge of the whole data set, it is not possible to identify a cluster that spans more than one partition by just looking at the partitions alone. The third phase of RDD-DBSCAN, global labeling, overcomes this limitation.

In the global labeling phase, RDD-DBSCAN will examine all points that are within ϵ of the borders of the bounding rectangle of each partition. If any of those points has been simultaneously assigned to more than one cluster, it follows that those clusters are the same (a more detailed discussion can be found in Section III-D). Once all global clusters have been identified in this way, all the points are relabeled with the correct global cluster and the algorithm concludes.

Fig. 1 gives an overview of the steps necessary to perform RDD-DBSCAN.

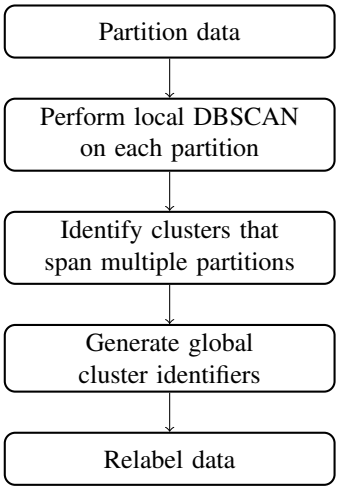


Fig. 1. RDD-DBSCAN overview

One important restriction of RDD-DBSCAN is that it assumes that the data points to be clustered can be represented in two dimensions. The reason for this restriction is that RDD-DBSCAN uses a two-dimension representation of the space which contains the data to come up with an efficient partitioning scheme. If the data cannot be represented in a two-dimension space, then partitioning fails. There is a wide body of literature dealing with dimensionality reduction, so it

Authorized licensed use limited to: UEL - Universidade Estadual de Londrina. Downloaded on October 21,2026 at 19:56:14 UTC from IEEE Xplore. Restrictions apply.

is expected that this restriction does not limit the applicability of RDD-DBSCAN.

We now present a more detailed discussion of each of the steps of the RDD-DBSCAN algorithm.

B. Data Partitioning

Data partitioning is the most critical step in the parallelization of DBSCAN. Partitions must be chosen carefully, so that the concurrent execution of the subsequent steps in the algorithm can become possible. Also, since one of the benefits of RDDs is the ability to make multiple passes over data that is already loaded into memory, care must be exercised in order to avoid creating partitions that exceed the available memory, forcing RDDs to be loaded from disk and negating the speed benefits that RDDs provide.

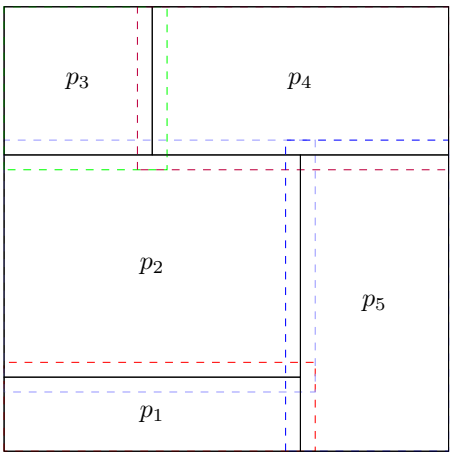


Fig. 2. A space partitioned by Algorithm 3

The first requirement for a partitioning strategy, is that all the points inside a partition should be able to participate in the DBSCAN algorithm without requiring knowledge of data in another partition. Y. He et al., proved in [14] that for a given a set of points bounded by the rectangle R, which is then further subdivided into two non-overlapping rectangles S_1, S_2, it is possible to execute DBSCAN independently in S_1, S_2 and obtain the same results as if DBSCAN was executed only on R, if the execution of DBSCAN on S_1, S_2 is made to include the *core* and *border* points that are within ϵ of the borders of S_1, S_2. The inclusion of the *core* and *border* points in the execution of DBSCAN on S_1, S_2 is necessary, because those points are used to identify clusters that span a larger area than the one covered by either S_1, S_2. For example, if the execution of DBSCAN in $S_1 + \epsilon$ identified a cluster C_1 and the execution of DBSCAN in $S_2 + \epsilon$ identified a cluster C_2, then there is a cluster C_3 that includes the points in C_1, C_2, if there exists a point that belongs to both C_1 and C_3 and there is a point that belongs to both C_3 and C_2. In this example, C_3 is the cluster that would have been found by the execution of DBSCAN over R. It then follows that, for the partitioning to fulfill the requirement that each partition must be able to be processed independently for a given minimum bounding rectangle BR that is split into a set of non-overlapping rectangles $\{S_1, S_2, \ldots, S_n\}$, a partition corresponding to S_n must also include the points that are within one ϵ of its borders. From the last statement it is also evident that the minimum size for S_n is 2ϵ, since any size smaller than that would result in overlapping partitions.

To find the optimum partitions for a particular set of data, we use binary space partitioning (BSP). BSP was introduced by M. Berger and S. Bokhari in [15]. BSP recursively splits the desired space into areas of roughly equal cost.

For RDD-DBSCAN, we use BSP to find the optimal partitions of the minimum bounding rectangle BR that includes the whole data set, with the following modifications: we introduce the $MinSize$ parameter, which restricts the size of the smaller partition, and the $MaxPoints$ parameter, which controls the maximum number of points that a partition may contain. Also, in RDD-DBSCAN we define cost as the number of points contained by each partition. The algorithm is presented in Algorithm 3.

Algorithm 3 EVENLYPARTITION

Input: For set of points $X = \{p_1, p_2, \ldots, p_n\}$, the minimum bounding rectangle BR for the points in X, the minimum size of a subdividing rectangle $MinSize$ and the maximum number of points in a rectangle $MaxPoints$.

Output: $R = \{r_1, r_2, \ldots, r_n\}$ where r_n are non-overlapping and $BR = \cup R_n$

```
1:  needingSplit ← {BR}
2:  partitions ← ∅
3:  foreach r ∈ needingSplit do
4:      if POINTSIN(r) ≥ MaxPoints and LENGTH(r) ≥ MinSize then
5:          {p1, p2} ← EVENLYSPLIT(r, MinSize)
6:          needingSplit ← needingSplit ∪ {p1, p2}
7:      else
8:          partitions ← partitions ∪ {current}
9:      end if
10: end for
11: return partitions
```

Algorithm 3 will recursively split the rectangle BR until one of the splits is either smaller than $MaxPoints$ or the split has reached $MinSize$.

$MaxPoints$ must be set to the maximum number of points that can fit into the memory of the machine with the lowest memory that is participating in the execution of RDD-DBSCAN. This restriction is necessary because the RDDs abstraction does not provide a way to pin execution of the algorithm to particular machine, thus all possible executions must fit in all machines.

Although the inclusion of the parameter $MaxPoints$ might seem like a limitation of RDD-DBSCAN, in practice, this parameter can be automatically deduced form the environment and is not a practical consideration for the user.

$MinSize$ on the other hand, becomes a fixed constant of 2ϵ in RDD-DBSCAN. As explained in the previous paragraphs,

Authorized licensed use limited to: UEL - Universidade Estadual de Londrina. Downloaded on October 21,2026 at 19:56:14 UTC from IEEE Xplore. Restrictions apply.

any partition smaller than 2ϵ would also be contained by some other partition, contradicting the purpose of partitioning making 2ϵ becomes the minimum size.

Finally, Algorithm 3 utilizes the procedure EVENLYSPLIT to find splits that are roughly equal to one another, dividing the space into splits that contain a similar number of points. The procedure EVENLYSPLIT is described in Algorithm 4.

Algorithm 4 EVENLYSPLIT

Input: A rectangle to split, S, and $MinSize$, the minimum size of a split.

Output: two rectangles S_1, S_2, where $S = S_1 \cup S_2, S_1 \cap S_2 = \emptyset$ and the points in S_1, S_2 are roughly equal.

1: $minimumDifference \leftarrow \infty$
2: $(S_1, S_2) \leftarrow (None, None)$
3: $candidates \leftarrow \{p_1, p_2, \ldots, p_n\}$ where $p_n = (c_1, c_2)$ and $c_1 \cup c_2 = S, c_1 \cap c_2 = \emptyset$ and $length(c_1) \geq length(c_2) \geq MinSize$
4: **foreach** $(C_1, C_2) \in candidates$ **do**
5: $difference \leftarrow |\text{POINTSIN}(C_1) - \text{POINTSIN}(C_2)|$
6: **if** $difference < minimumDifference$ **then**
7: $minimumDifference \leftarrow difference$
8: $(S_1, S_2) \leftarrow (C_1, C_2)$
9: **end if**
10: **end for**
11: **return** (S_1, S_2)

Algorithm 4 returns a pair of splits that have roughly the same number of points in each split. The approach of using the number of points as the criteria for a split differs from the approach presented in [14], which uses the approximate cost of executing DBSCAN on each partition as the driving criteria. RDD-DBSCAN cannot use the approach given in [14] to calculate the cost, because that approach assumes that disk access is one of the factors that affects the cost.

RDD-DBSCAN, however, does most of the data access in memory, so the cost of disk access for the data is not taken into account. Counting the number of points in each partition provides a sufficiently good indicator of a good split.

Once the rectangles that evenly split the minimum bounding rectangle BR for the data space have been found, each one of those rectangles is grown by ϵ. This particular modification of the rectangles is done in the main RDD-DBSCAN algorithm, specifically Line 5 in Algorithm 2. The set of points contained by each of those rectangles corresponds to a partition.

In summary, the result of the data partitioning stage is a set of partitions $P = \{p_1, p_2, \ldots, p_n\}$, such that each p_n has roughly the same number of points and where each p_n overlaps by ϵ with its neighboring partitions. The result of the execution of Algorithm 3 can be seen in Fig. 2.

C. Local DBSCAN

After the data partitioning stage has completed, it is possible to run the traditional DBSCAN algorithm on each partition (Algorithm 1). Since the previous partitioning method ensured that no partition had more points, than those that could fit into the memory of any single machine participating in RDD-DBSCAN, any sequential variation of DBSCAN can be used to perform this step by any of the participating machines.

One important point to notice is that DBSCAN will generate cluster identifiers that are unique only to their partitions. That is, it is possible that different clusters in different partitions will share the same cluster identifier. However, this does not mean that both clusters represent a single one. To prevent ambiguity, at this point, a cluster should be identified by not only its cluster identifier (as returned by the local execution of DBSCAN), but by the pair of both the cluster identifier and the partition identifier.

The output of the local DBSCAN stage of RDD-DBSCAN is a set of labeled points, where each point has been labeled as one of *core*, *border*, or *noise*, and in the case of of *core* and *border* points a cluster identifier has been assigned to such points.

D. Global Labeling

The goal of the global labeling phase is to identify which of the previously found local cluster identifiers correspond to a single cluster that spans more than one partition. Also, global cluster identifiers should be generated from the local cluster identifiers in order to provide an identical output to the one produced by the local DBSCAN algorithm. Once these two steps have been done, all points need to be relabeled using the newly-found, globally-unique identifiers.

Fig. 3 illustrates a situation in which two clusters should be merged. In this particular case, the partitioning stage divided the data space into two partitions represented respectively by the p_1 and p_2. Note that the dotted lines represent the expansion of ϵ that was done for each partition; that is partition p_1 includes the space within the green dotted line, and partition p_2 includes the space within the red dotted line. The points that are within the dotted lines belong to both partitions.

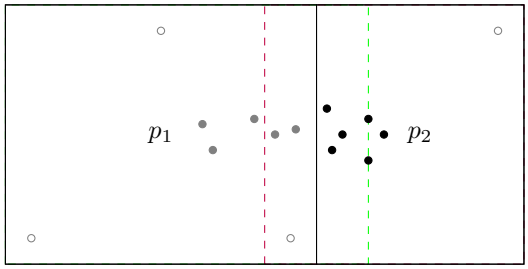


Fig. 3. Clusters that are candidates for merging

The execution of the local DBSCAN found in each partition found a few *noise* points, represented by the empty circles, and also found a cluster in each partition, represented by the grey and black points respectively. However, if DBSCAN would have been executed including the points for both partitions, both the grey and black clusters would have been identified as a single one. It becomes necessary then to find a way to find local clusters that are part (or aliases) for another larger global

Authorized licensed use limited to: UEL - Universidade Estadual de Londrina. Downloaded on October 21,2026 at 19:56:14 UTC from IEEE Xplore. Restrictions apply.

cluster. We present the algorithm to find clusters identifiers that are aliases for the same global cluster in Algorithm 5.

Algorithm 5 Procedure IDENTIFYCLUSTERALIASES

Input: $Par = p_1, p_2, \ldots, p_n$ where p_n overlaps by ϵ with its neighbors, the result of Algorithm 1, $LabeledPoints = \{p_1, p_2, \ldots, p_n\}$ and the distance threshold ϵ
Output: $\{(c_1, c_2)_1, (c_1, c_2)_2, \ldots, (c_1, c_2)_n\}$ where c_1, c_2 are aliases for the same cluster

```
1:  C ← ∅
2:  foreach partition ∈ par do
3:      partition ← shrink partition by ε
4:      Cs ← ∅
5:      points ← ∅
6:      foreach point ∈ partition and point within ε of partition's border and point not noise do
7:          if point ∈ points then
8:              Cs ← (point.label, prevpoint.label)
9:          else
10:             points ← points ∪ {point}
11:         end if
12:     end for
13:     C ← C ∪ Cs
14: end for
15: return C
```

The algorithm first shrinks each partition, so that each partition does not overlap with any other one. Then it filters out all points that are not within ϵ of the border of the partition and it also filters out the points that are noise. Filtering out is not strictly necessary, but doing the filtering prevents unnecessary data transfer and computing costs.

Then, for each partition, the algorithm examines all the points that belong to the space contained by the partition. Upon examination some points may have multiple labels, since they may have previously been part of a different partition. It is those points that are relevant for our purposes. Each label, of a point that has multiple labels, is an alias for the same global cluster.

The result of executing the previous step, over all points in all partitions, is a list of aliases for global clusters. If we return to our example, the result of executing Algorithm 5 on the pictured data set, would be the set of aliases $Aliases = \{(p_1, gray), (p_2, black)\}$.

Once all aliases have been found, we need to collect a list of all the clusters that are present in the data set and assign to them a globally unique identifier. This step is not strictly necessary, but it is performed in order to produce an identical result to the result of executing the original DBSCAN algorithm. After we have the list of globally unique identifiers, all labeled points are relabeled with the globally unique identifier.

One particular detail that may not be obvious at first glance is that, since a single point can have multiple labels, a single point can be both *border* and *core* at the same time. If this is the case, the point is labeled as a *core* point.

We present the algorithm to execute the relabeling in Algorithm 6. This algorithm closely follows Algorithm 8 presented by H. Yaobin et al. in [14]. Note that the section that finds the globally unique identifiers is not parallelized. We chose to present a sequential algorithm here in the name of simplicity, however parallelizing this section, if proved necessary, should be trivial.

Algorithm 6 Procedure RELABELPOINTS

Input: $LabelPairs = \{l_1, l_2, \ldots, l_n\}$ where l_n is a pair of $(partition, cluster)$, $Aliases = \{a_1, a_2 \ldots, a_n\}$ where a is a pair of labels (l_1, l_2), and $LabeledPoints = \{p_1, p_2, \ldots, p_n\}$
Output: $LabeledPoints = \{p_1, p_2, \ldots, p_n\}$

```
1:  id ← next identifier
2:  g ← empty graph
3:  mapping ← empty map
4:  foreach (l1, l2) ∈ Aliases do
5:      if vertex l1 not in g then
6:          add vertex l1 to g
7:      end if
8:      if vertex l2 not in g then
9:          add vertex l2 to g
10:     end if
11:     if edge (l1, l2) is not in g then
12:         add edge (p1, p2) to g
13:     end if
14: end for
15: foreach connected vertex v in g do
16:     if v ∉ mapping then
17:         insert (v, id) in mapping
18:         id ← next identifier
19:     end if
20: end for
21: foreach label ∈ LabelPairs do
22:     if label ∉ mapping then
23:         insert (label, id) in mapping
24:         id ← next identifier
25:     end if
26: end for
27: foreach p ∈ LabeledPoints do
28:     if (p.partition, p.label) ∈ mapping then
29:         p.label ← mapping[(p.partition, p.label)]
30:     end if
31: end for
32: return LabeledPoints
```

IV. EVALUATION

A. Platform

To evaluate the performance and correctness of RDD-DBSCAN we chose to implement RDD-DBSCAN using Apache Spark, the popular implementation of the RDDs abstraction. Since its introduction, Apache Spark has become extremely popular and has had tremendous growth: as of 2014, Apache Spark is the most active open source project in the Big

Authorized licensed use limited to: UEL - Universidade Estadual de Londrina. Downloaded on October 21,2026 at 19:56:14 UTC from IEEE Xplore. Restrictions apply.

Data ecosystem [16]. As such, Apache Spark is the obvious target for the implementation of RDD-DBSCAN.

B. Language

Apache Spark is implemented in Scala but it also has bindings to Java and Python, allowing for algorithms to be implemented in any of those languages. We chose to implement RDD-DBSCAN using Scala, because Apache Spark itself is written in this language; using Scala provided the best interoperability with the platform.

C. Test Data

A data set is an integral part of the evaluation of clustering algorithms, and for the evaluation of RDD-DBSCAN we chose to use a synthetic data set. A synthetic data set allowed us to better observe the behavior of RDD-DBSCAN under different conditions that are harder to control in non-synthetic data. The synthetic data was generated with *scikit-learn*'s [17] *samples_generator* utility [18]. The data set consisted of one million entries, organized into five different clusters with five different centers.

D. Correctness

Since DBSCAN is mostly deterministic, it is possible to verify that a distributed version of DBSCAN is correct by comparing the results of both the original version and the distributed version. If the results of both are identical, then the distributed version is correct.

To verify the correctness of RDD-DBSCAN, we generated a smaller version of the test data with the tools mentioned in the previous paragraph. The test data was then clustered using *scikit-learn*'s implementation of DBSCAN and with RDD-DBSCAN. The output of both executions of the algorithm was identical, so we were able to verify that RDD-DBSCAN was correct.

E. Complexity

Given that DBSCAN, as described in Algorithm 1, has a complexity of $\mathcal{O}(n^2)$ we chose not to use that algorithm in our local DBSCAN phase. Instead we used an R-tree [19] as a helping data structure to perform the GETNEIGHBORS query in Line 13 of the algorithm. With the help of R-tree the complexity of local DBSCAN comes down to $\mathcal{O}(n \log n)$.

One important observation is that the R-tree used in our implementation is completely held in memory, with each R-tree containing all the points for any given partition. Holding the R-tree in memory allows for very fast execution of the algorithm, but it constrains the number of points that may be included in a partition, to those that can be held in memory. In our experiments, exceeding the memory capacity of our nodes resulted in the Java Virtual Machine entering long garbage collection cycles and eventually crashing with out of memory errors.

Apache Spark exposes the amount of memory available for the execution of a given algorithm, so it is possible to automatically detect this condition before it happens. However, for a large enough value of ϵ, a partition might exceed this value regardless. The fix, in this particular case, is to select a smaller ϵ so that the partition will fit into memory.

F. Hardware

Our implementation of RDD-DBSCAN was evaluated using a cluster of five virtual machines running inside the Amazon Web Services environment. Each machine was an instance of an *m3.large* model, with each instance having 2 vCPUs of type Intel Xeon E5-2670 v2, 7.5 GiB of memory and 32 GB of SSD Storage.

G. Results

The most important goal of our experiment, was to determine whether RDD-DBSCAN would scale linearly as the amount of available computing power was increased. Fig. 4 confirmed that, as expected, the time taken by RDD-DBSCAN decreased as the number of cores available for computation increased.

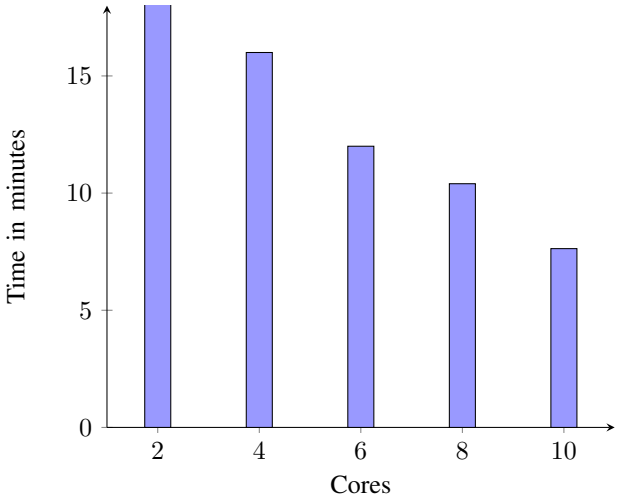


Fig. 4. Total time vs. number of cores

To understand why this is the case, it is worthwhile to understand how Apache Spark allocates work within a cluster: first, Apache Spark splits the pending work into tasks, where a task is the work that has to be done for a single partition. Then, each computing node, or worker, is assigned to one of these tasks, until all workers are busy; with each worker only working on a predetermined number of tasks at any given time. This number is usually set to the number of CPUs available per worker. The remaining tasks are then placed on a queue, and they are be taken by the workers, as the workers complete their previous tasks. From that, it becomes obvious that as the number of computing nodes increases the parallelism increases, as more partitions can be processed at the same time.

One important drawback of distributed algorithms is that, as their parallelism increases, so does the cost of communication between the computing nodes. If the cost in communication is not kept in check the cost of communication may end up

Authorized licensed use limited to: UEL - Universidade Estadual de Londrina. Downloaded on October 21,2026 at 19:56:14 UTC from IEEE Xplore. Restrictions apply.

dominating the cost of computing, diminishing the gains made from parallelization.

To determine whether RDD-DBSCAN suffered this drawback, we measured the costs of communication and the amount of data transported between nodes as the number of nodes increased, the results are shown in Fig. 5.

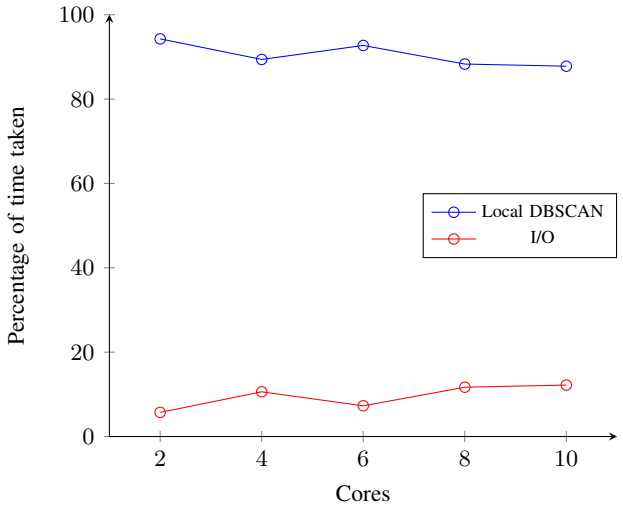
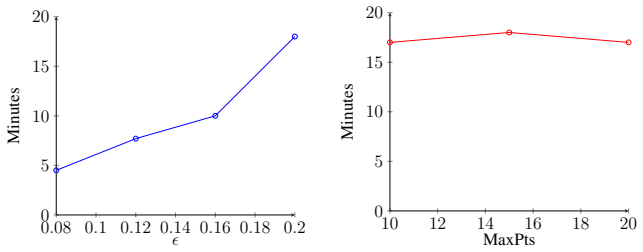


Fig. 5. Time spent in each phase of the algorithm

Our experiments show that first, the time taken by computing tasks completely dominates the costs of communication between nodes; and second, that as the number of nodes increases, the percentage of time taken by the computing tasks remains constant. These results indicate that RDD-DBSCAN is not significantly affected by communication costs, which is not surprising given that Apache Spark only exposes a very limited set of intra-node communication mechanisms.

Our final experiment consisted of determining the influence that the different parameters of RDD-DBSCAN had on its running time. The results of our experiments are summarized in Fig. 6.



(a) ϵ (b) *MaxPoints*

Fig. 6. The impact in running time for different parameters

The first important result is that, as the size of ϵ increases, so does the execution time of the algorithm. The reason behind this increase is that a larger value of ϵ forces the algorithm to create larger partitions (the minimum size of a partition is limited by the ϵ value). Larger partitions, in turn, tend to contain a larger number of points which have to be loaded into memory by the computing node when they are processed. For our particular implementation of the local DBSCAN algorithm, the larger the number of points that have to be processed at the same time, the poorest the performance.

To figure out whether it was possible to influence the execution time of a large ϵ value by varying the size of the minimum partition, we kept the same value for ϵ (in this particular case $\epsilon = 0.2$) but varied the value for $MaxPoints$. From our experiments, varying the size of $MaxPoints$ did not particularly affect the execution time of RDD-DBSCAN. Different values of $MaxPoints$ did not affect the quality of the partitions, because some partitions could not be improved due to the limitations set by the large value of ϵ.

V. Conclusion

This paper introduced RDD-DBSCAN, a distributed version of the DBSCAN algorithm using Resilient Distributed Datasets that produces an identical outcome to that of the original DBSCAN. We described the considerations that go into the design of the algorithm, as well as the advantages of this approach over other similar approaches. We provided a detailed explanation of each step of the algorithm and also described how these steps can be translated into an actual implementation. Then, we proved experimentally that RDD-DBSCAN scales as the number of nodes in a given cluster scale while generating the same results as the sequential version of DBSCAN.

VI. Future Work

The main limitation of RDD-DBSCAN is the need to load the complete data set for a given partition into memory. Future work could focus in removing this limitation from RDD-DBSCAN. This could be done through allowing the user to provide a flag, that when set, would allow for portions of the R-tree (which holds the data for the partition), to be offloaded to disk and then loaded when needed. This approach would be similar to the one already offered by Apache Spark, where Apache Spark provides different storage levels for the RDDs. In particular, this approach would be analogous to the MEMORY_AND_DISK storage level provided by Apache Spark.

Another area for improvement, is the selection of a better partitioning scheme for the data space. In our approach we deemed the number of points per partition as a sufficiently good condition for determining good partitions. However, in heavily skewed data, this assumption may not be correct and a better approach may be found.

The partitioning scheme may also be improved by extending it to work into more than two dimensions. Many sets of data that need clustering cannot be adequately represented by just two dimensions. Improving the partitioning scheme so that it may successfully partition *n-dimensional* data would allow RDD-DBSCAN to cluster more diverse data sets.

Authorized licensed use limited to: UEL - Universidade Estadual de Londrina. Downloaded on October 21,2026 at 19:56:14 UTC from IEEE Xplore. Restrictions apply.

REFERENCES

[1] (2014, August 18) 2014 SIGKDD Test of Time Award. [Online]. Available: http://www.kdd.org/blog/2014-sigkdd-test-time-award

[2] M. Ester, H.-P. Kriegel, J. Sander, and X. Xu, "A density-based algorithm for discovering clusters in large spatial databases with noise." in *KDD*, vol. 96, no. 34, 1996, pp. 226–231.

[3] M. Chen, X. Gao, and H. Li, "Parallel dbscan with priority r-tree," in *Information Management and Engineering (ICIME), 2010 The 2nd IEEE International Conference on*. IEEE, 2010, pp. 508–511.

[4] D. Arlia and M. Coppola, "Experiments in parallel clustering with dbscan," in *Euro-Par 2001 Parallel Processing*. Springer, 2001, pp. 326–331.

[5] M. M. A. Patwary, D. Palsetia, A. Agrawal, W.-k. Liao, F. Manne, and A. Choudhary, "A new scalable parallel dbscan algorithm using the disjoint-set data structure," in *High Performance Computing, Networking, Storage and Analysis (SC), 2012 International Conference for*. IEEE, 2012, pp. 1–11.

[6] J. Dean and S. Ghemawat, "Mapreduce: simplified data processing on large clusters," *Communications of the ACM*, vol. 51, no. 1, pp. 107–113, 2008.

[7] Wikipedia. (2015, April, 4) Mapreduce — Wikipedia, the free encyclopedia. Accessed 22-July-2004. [Online]. Available: http://en.wikipedia.org/wiki/MapReduce

[8] B.-R. Dai and I.-C. Lin, "Efficient map/reduce-based dbscan algorithm with optimized data partition," in *Cloud Computing (CLOUD), 2012 IEEE 5th International Conference on*. IEEE, 2012, pp. 59–66.

[9] Y. He, H. Tan, W. Luo, H. Mao, D. Ma, S. Feng, and J. Fan, "Mr-dbscan: an efficient parallel density-based clustering algorithm using mapreduce," in *Parallel and Distributed Systems (ICPADS), 2011 IEEE 17th International Conference on*. IEEE, 2011, pp. 473–480.

[10] M. Zaharia, M. Chowdhury, T. Das, A. Dave, J. Ma, M. McCauley, M. J. Franklin, S. Shenker, and I. Stoica, "Resilient distributed datasets: A fault-tolerant abstraction for in-memory cluster computing," in *Proceedings of the 9th USENIX conference on Networked Systems Design and Implementation*. USENIX Association, 2012, pp. 2–2.

[11] Wikipedia. (2015, April, 5) Scalability — Wikipedia, the free encyclopedia. Accessed 22-July-2004. [Online]. Available: http://en.wikipedia.org/wiki/Scalability

[12] K.-H. Lee, Y.-J. Lee, H. Choi, Y. D. Chung, and B. Moon, "Parallel data processing with mapreduce: a survey," *AcM sIGMoD Record*, vol. 40, no. 4, pp. 11–20, 2012.

[13] Y. X. Fu, W. Z. Zhao, and H. F. Ma, "Research on parallel dbscan algorithm design based on mapreduce," *Advanced Materials Research*, vol. 301, pp. 1133–1138, 2011.

[14] Y. He, H. Tan, W. Luo, S. Feng, and J. Fan, "Mr-dbscan: a scalable mapreduce-based dbscan algorithm for heavily skewed data," *Frontiers of Computer Science*, vol. 8, no. 1, pp. 83–99, 2014.

[15] M. J. Berger and S. H. Bokhari, "A partitioning strategy for nonuniform problems on multiprocessors," *Computers, IEEE Transactions on*, vol. 100, no. 5, pp. 570–580, 1987.

[16] P. Wendell and M. Zaharai. (2015, February 13) Spark: A review of 2014 and looking ahead to 2015 priorities. [Online]. Available: https://databricks.com/blog/2015/02/13/spark-a-review-of-2014-and-looking-ahead-to-2015-priorities.html

[17] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[18] (2015, April 12) Dataset loading utilities. [Online]. Available: http://scikit-learn.org/stable/datasets/

[19] A. Guttman, *R-trees: a dynamic index structure for spatial searching*. ACM, 1984, vol. 14, no. 2.

Authorized licensed use limited to: UEL - Universidade Estadual de Londrina. Downloaded on October 21,2026 at 19:56:14 UTC from IEEE Xplore. Restrictions apply.